

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

EXPERIMENT 2 - NODE.JS FILE MANAGEMENT USING THE FILE SYSTEM MODULE (CO-2)

1. TITLE

Develop a Node.js Application to Perform File Creation, Reading, Appending, and Display Operations Using the File System (fs) Module.

2. AIM

To design and implement a Node.js console application that accepts a filename and content from the user through the terminal and performs the following operations using the built-in File System module: create/write the file, read its contents, append additional content, and display the final contents of the file.

3. OBJECTIVE

- To understand the architecture of Node.js and the use of built-in core modules through `require()`.
- To accept interactive user input from the terminal using the `readline` module.
- To create and write a file on the server file system using `fs.writeFileSync()`.
- To read the contents of a file using `fs.readFileSync()` with UTF-8 encoding.
- To append additional data to an existing file without overwriting it using `fs.appendFileSync()`.
- To implement input validation and structured error handling with `try...catch`.
- To differentiate between synchronous (blocking) and asynchronous (non-blocking) file operations.

4. THEORY

1. Node.js Runtime Environment

Node.js is an open-source, cross-platform JavaScript runtime built on Google Chrome's V8 engine. It allows JavaScript to be executed outside the browser, on the server side. Node.js follows a single-threaded, event-driven, non-blocking I/O model which makes it efficient for data-intensive applications.

2. Modules in Node.js

A module is a reusable block of code. Node.js provides **core (built-in) modules** such as `fs`, `path`, `http`, `os`, and `readline`, which are loaded using the CommonJS function `require('moduleName')`.

3. The File System (fs) Module

The `fs` module provides an API for interacting with the file system. Every method is available in two forms:

- **Synchronous (blocking):** `fs.writeFileSync()`, `fs.readFileSync()`, `fs.appendFileSync()`. The execution of the program is paused until the operation finishes, and the result is returned directly.

- **Asynchronous (non-blocking):** `fs.writeFile()`, `fs.readFile()`, `fs.appendFile()`. These accept an error-first callback of the form `(err, data) => { }` and allow the event loop to continue.

4. Important fs Methods Used

- `fs.writeFileSync(path, data, encoding)` - creates the file if it does not exist, and **overwrites** it if it already exists.
- `fs.readFileSync(path, 'utf8')` - reads the entire file and returns a string. Without the encoding argument, a raw Buffer object is returned instead.
- `fs.appendFileSync(path, data, encoding)` - adds data to the end of the file, preserving the existing contents.
- `fs.existsSync(path)` - returns a boolean indicating whether the path exists.
- `fs.mkdirSync(path)` - creates a new directory.
- `fs.statSync(path)` - returns a Stats object containing metadata such as file size and timestamps.

5. The readline Module

The readline module reads data from a readable stream line by line. `readline.createInterface({ input: process.stdin, output: process.stdout })` creates an interface bound to the terminal, and `rl.question(prompt, callback)` displays a prompt and captures the user's answer. In this program the callback is wrapped in a Promise so that `async/await` can be used for readable sequential logic.

6. The path Module

`path.join()` joins path segments using the correct separator for the operating system, and `path.extname()` returns the extension of a file name. Using these avoids hard-coded Windows or Linux path separators.

7. Error Handling

Synchronous fs methods throw exceptions on failure, so they are enclosed in a `try...catch` block. Common errors include `ENOENT` (no such file or directory), `EACCES` (permission denied), and `EISDIR` (the path is a directory).

5. ALGORITHM / PROCEDURE

1. Start.
2. Import the core modules `fs`, `path`, and `readline` using `require()`.
3. Create a readline interface bound to `process.stdin` and `process.stdout`.
4. Define a helper function `ask()` that wraps `rl.question()` inside a Promise.
5. Check whether the storage directory exists using `fs.existsSync()`; if not, create it with `fs.mkdirSync()`.
6. Prompt the user for a file name and validate it (non-empty, no path separators). If invalid, display an error message and terminate.
7. If no extension is present, append the `.txt` extension and build the full path using `path.join()`.
8. Prompt the user for the content and validate that it is not empty.
9. Write the content to the file using `fs.writeFileSync()`.
0. Read the file using `fs.readFileSync()` and display the current contents.
1. Prompt the user for additional content and append it using `fs.appendFileSync()`.
2. Read the file again and display the final contents along with the file size obtained from `fs.statSync()`.
3. Catch and display any file-system error inside the catch block.
4. Close the readline interface and stop.

6. PROGRAM

fileManager.js

```
/**
 * fileManager.js
 * Node.js File Management Application
 * Accepts a filename and content from the user through the terminal and performs
 * CREATE / WRITE -> READ -> APPEND -> READ FINAL operations using the fs module.
 */

const fs = require('fs');          // Built-in File System module
const path = require('path');      // Built-in Path module
const readline = require('readline');

// Create the interactive input interface (STDIN / STDOUT)
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Directory in which every user file will be stored
const STORAGE_DIR = path.join(__dirname, 'files');

// Promise wrapper around rl.question() so that we can use async/await
function ask(query) {
  return new Promise((resolve) => rl.question(query, (answer) => resolve(answer)));
}

// Ensure the storage folder exists before any write operation
function ensureStorageDir() {
  if (!fs.existsSync(STORAGE_DIR)) {
    fs.mkdirSync(STORAGE_DIR);
    console.log('Storage folder created at: ' + STORAGE_DIR);
  }
}

// Simple validation for the filename supplied by the user
function isValidFileName(name) {
  if (!name || name.trim() === '') return false;
  // Reject path separators so the user cannot escape the storage folder
  if (name.includes('/') || name.includes('\\')) return false;
  return true;
}

async function main() {
  console.log('=====');
  console.log('      NODE.JS FILE MANAGEMENT APPLICATION      ');
  console.log('=====');

  ensureStorageDir();

  // ----- STEP 1 : Collect input from the user -----
  let fileName = await ask('Enter the file name (example: notes.txt) : ');
  fileName = fileName.trim();

  if (!isValidFileName(fileName)) {
    console.error('ERROR: Invalid file name. Please avoid empty names and slashes. ');
    rl.close();
    return;
  }

  // Add a .txt extension if the user did not supply one
  if (path.extname(fileName) === '') {
    fileName = fileName + '.txt';
  }

  const filePath = path.join(STORAGE_DIR, fileName);

  const content = await ask('Enter the content to write : ');
  if (content.trim() === '') {
    console.error('ERROR: Content cannot be empty. ');
    rl.close();
    return;
  }

  try {
    // ----- STEP 2 : CREATE / WRITE the file -----
    fs.writeFileSync(filePath, content + '\n', 'utf8');
    console.log('\n[1] WRITE : File created successfully -> ' + fileName);

    // ----- STEP 3 : READ the file -----
    const initialData = fs.readFileSync(filePath, 'utf8');
    console.log('[2] READ : Current contents of the file : ');
    console.log('-----');
    console.log(initialData.trim());
    console.log('-----');

    // ----- STEP 4 : APPEND additional content -----
    const extra = await ask('Enter additional content to append : ');
    fs.appendFileSync(filePath, extra + '\n', 'utf8');
    console.log('[3] APPEND : Additional content appended successfully. ');

    // ----- STEP 5 : DISPLAY the final contents -----
    const finalData = fs.readFileSync(filePath, 'utf8');
    console.log('[4] FINAL CONTENTS OF ' + fileName + ' : ');
    console.log('=====');
    console.log(finalData.trim());
    console.log('=====');
```

```
// File statistics using fs.statSync()
const stats = fs.statSync(filePath);
console.log('File size   : ' + stats.size + ' bytes');
console.log('Location    : ' + filePath);

} catch (err) {
  // Centralised error handling for every fs operation
  console.error('FILE OPERATION FAILED : ' + err.message);
} finally {
  rl.close();
}
}

main();
```

asyncDemo.js (asynchronous version)

```
/**
 * asyncDemo.js (optional extension)
 * The same operations written with asynchronous, non-blocking callbacks.
 */

const fs = require('fs');

fs.writeFile('demo.txt', 'First line written asynchronously\n', 'utf8', (err) => {
  if (err) return console.error('Write failed: ' + err.message);
  console.log('Write completed (non-blocking)');

  fs.appendFile('demo.txt', 'Second line appended asynchronously\n', 'utf8', (err2) => {
    if (err2) return console.error('Append failed: ' + err2.message);
    console.log('Append completed (non-blocking)');

    fs.readFile('demo.txt', 'utf8', (err3, data) => {
      if (err3) return console.error('Read failed: ' + err3.message);
      console.log('Final contents:\n' + data);
    });
  });
});
```

Command to Execute

```
$ node fileManager.js
```

7. CODE EXPLANATION

Code	Explanation
<code>require('fs')</code>	Imports the built-in File System core module that exposes all file input/output methods.
<code>require('path')</code>	Imports the Path module, used to build operating-system independent file paths.
<code>readline.createInterface({...})</code>	Creates an interactive input/output interface bound to the terminal streams.
<code>rl.question(prompt, cb)</code>	Displays a prompt on the terminal and passes the user's typed answer to the callback.
<code>new Promise((resolve) => ...)</code>	Wraps the callback-based question method so that <code>async/await</code> can be used.
<code>fs.existsSync(STORAGE_DIR)</code>	Checks synchronously whether the storage directory already exists.
<code>fs.mkdirSync(STORAGE_DIR)</code>	Creates the storage directory when it is missing.
<code>path.join(__dirname, 'files')</code>	Joins the current script directory with the sub-folder using the correct separator.
<code>path.extname(fileName)</code>	Returns the file extension so that a default <code>.txt</code> extension can be added.
<code>fs.writeFileSync(path, data, 'utf8')</code>	Creates the file and writes the content; an existing file of the same name is overwritten.
<code>fs.readFileSync(path, 'utf8')</code>	Reads the complete file and returns it as a UTF-8 string instead of a Buffer.
<code>fs.appendFileSync(path, data, 'utf8')</code>	Appends new content at the end of the file without deleting the earlier content.
<code>fs.statSync(path).size</code>	Retrieves file metadata; the <code>size</code> property gives the file size in bytes.
<code>try { } catch (err) { }</code>	Handles exceptions thrown by synchronous <code>fs</code> methods and prints a readable error message.
<code>rl.close()</code>	Closes the readline interface and releases the terminal so the process can exit.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the application from the terminal.

Test Case	TC-01
Test / Input	Valid input: filename = student.txt, content = “Full Stack Web Development Lab Record”, appended text = “Node.js File System Module Demonstration”.
Expected Result	The file is created inside the files folder, the initial content is displayed, the appended line is added, and both lines are printed as the final contents.
Actual Result	File created, read, appended, and the final two-line content displayed correctly along with the file size.
Status	Pass

TC-01 Terminal Output:

```

Terminal - node fileManager.js

$ node fileManager.js
=====
      NODE.JS FILE MANAGEMENT APPLICATION
=====
Storage folder created at: /home/claude/run/task2/files
Enter the file name (example: notes.txt) : student.txt

Enter the content to write : Full Stack Web Development Lab Record

[1] WRITE  : File created successfully -> student.txt
[2] READ   : Current contents of the file :
-----
Full Stack Web Development Lab Record
-----
Enter additional content to append : Node.js File System Module Demonstration

[3] APPEND : Additional content appended successfully.
[4] FINAL CONTENTS OF student.txt :
=====
Full Stack Web Development Lab Record
Node.js File System Module Demonstration
=====
File size  : 79 bytes
Location   : /home/claude/run/task2/files/student.txt
```

Test Case	TC-02
Test / Input	Invalid input: the filename is left blank and the Enter key is pressed.
Expected Result	The application rejects the input and displays the message “ERROR: Invalid file name.” without creating any file.
Actual Result	Validation triggered correctly, no file was created, and the process exited cleanly.
Status	Pass

TC-02 Terminal Output:

```

Terminal - node fileManager.js

$ node fileManager.js
=====
      NODE.JS FILE MANAGEMENT APPLICATION
=====
Enter the file name (example: notes.txt) :

ERROR: Invalid file name. Please avoid empty names and slashes.
```

Test Case	TC-03
Test / Input	Existing file: the program is executed a second time with the same filename student.txt.
Expected Result	fs.writeFileSync() overwrites the previous contents, and the appended line is added to the new content only.
Actual Result	The old content was replaced as expected, confirming the overwrite behaviour of writeFileSync().
Status	Pass

TC-03 Terminal Output:

Terminal - node fileManager.js

```
$ node fileManager.js
=====
      NODE.JS FILE MANAGEMENT APPLICATION
=====
Enter the file name (example: notes.txt) : student.txt

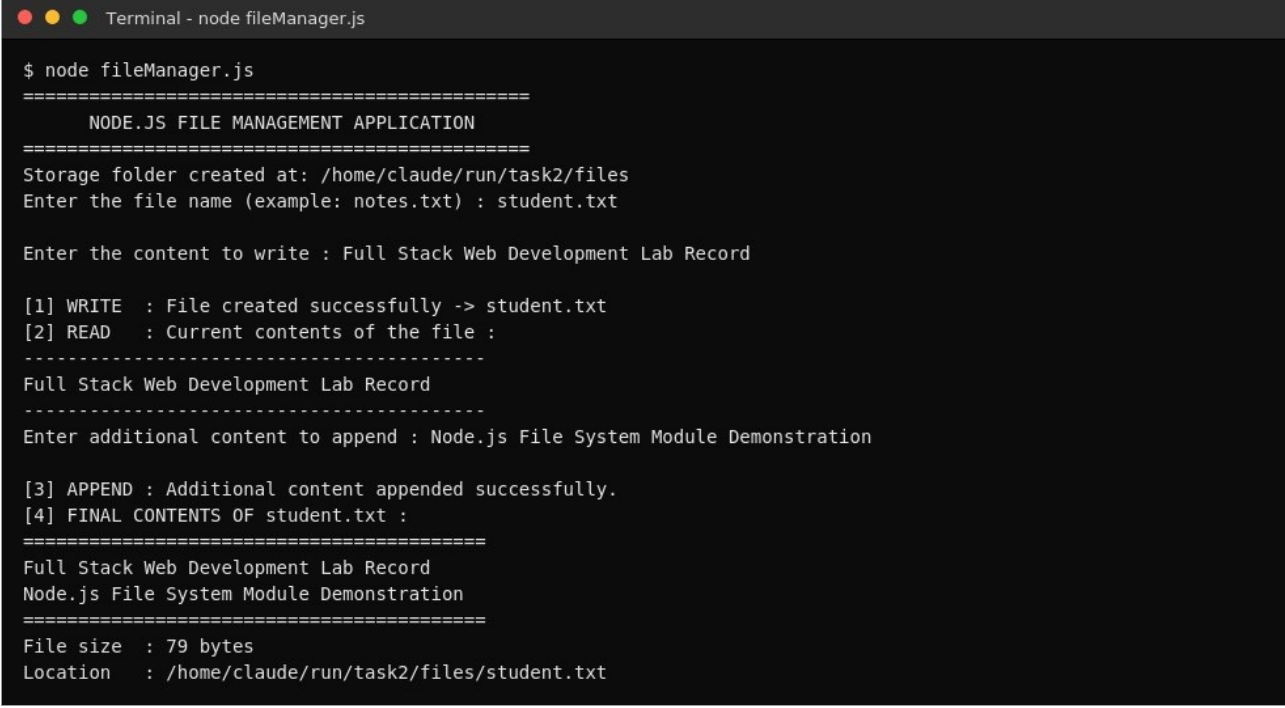
Enter the content to write : Second run - overwrite test

[1] WRITE  : File created successfully -> student.txt
[2] READ   : Current contents of the file :
-----
Second run - overwrite test
-----
Enter additional content to append : appended in second run

[3] APPEND : Additional content appended successfully.
[4] FINAL CONTENTS OF student.txt :
=====
Second run - overwrite test
appended in second run
=====
File size   : 51 bytes
Location    : /home/claude/run/task2/files/student.txt
```

9. OUTPUT

Output 1: Node.js File Management Application executed in the terminal.

A terminal window titled "Terminal - node fileManager.js" showing the execution of a Node.js application. The user runs the command "\$ node fileManager.js". The application outputs a header "NODE.JS FILE MANAGEMENT APPLICATION" and a separator line. It then displays the storage folder path "/home/claude/run/task2/files" and prompts for a file name. The user enters "student.txt". Next, it prompts for content to write, and the user enters "Full Stack Web Development Lab Record". The application confirms the file was created successfully. It then prompts for the current contents, and the user enters the same text. The application displays the current contents and prompts for additional content to append. The user enters "Node.js File System Module Demonstration". The application confirms the content was appended successfully. Finally, it displays the final contents of the file, which is the concatenation of the two inputs. It also shows the file size as 79 bytes and the location as "/home/claude/run/task2/files/student.txt".

```
$ node fileManager.js
=====
      NODE.JS FILE MANAGEMENT APPLICATION
=====
Storage folder created at: /home/claude/run/task2/files
Enter the file name (example: notes.txt) : student.txt

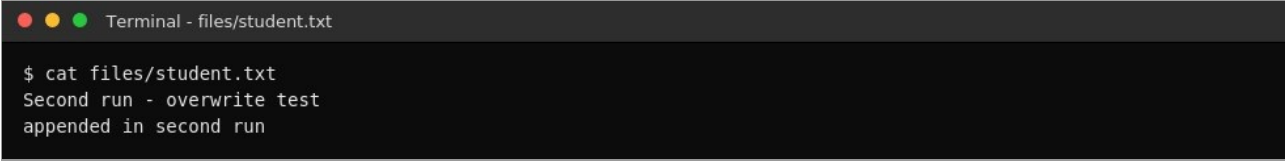
Enter the content to write : Full Stack Web Development Lab Record

[1] WRITE  : File created successfully -> student.txt
[2] READ   : Current contents of the file :
-----
Full Stack Web Development Lab Record
-----
Enter additional content to append : Node.js File System Module Demonstration

[3] APPEND : Additional content appended successfully.
[4] FINAL CONTENTS OF student.txt :
=====
Full Stack Web Development Lab Record
Node.js File System Module Demonstration
=====
File size  : 79 bytes
Location   : /home/claude/run/task2/files/student.txt
```

Figure 1: Terminal Output of the Node.js File Management Application

Output 2: The generated file opened in a text editor.

A terminal window titled "Terminal - files/student.txt" showing the contents of the file created by the application. The user runs the command "\$ cat files/student.txt". The output displays the two lines of text that were written and appended: "Second run - overwrite test" and "appended in second run".

```
$ cat files/student.txt
Second run - overwrite test
appended in second run
```

Figure 2: Final Contents of the Generated File

10. RESULT

Thus, the **Node.js File Management Application** was successfully developed and executed. The application accepted a filename and content from the user and performed create/write, read, append, and final display operations using the File System module, with proper input validation and error handling.